

AI & ML for Data Scientists

Ensemble Learning

Ray Ge

Quant RUC (人大量化) & IE Finance

February 17, 2026



Base Knowledge

Ensemble Models

Ensemble Model Framework

Ensemble algorithm paper (Chen 2016 XgBoost)

Base Knowledge

Ensemble Models

Ensemble Model Framework

Ensemble algorithm paper (Chen 2016 XgBoost)

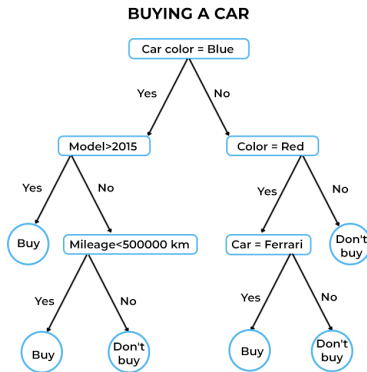
Base Knowledge for Ensemble

- Decision Tree (Previous Class)
- Voting
- Bootstrap
- Bagging

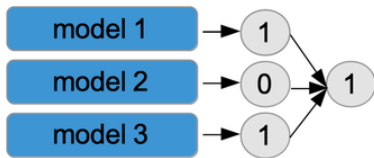
```
graph TD; A[Salary Range  
$50,000 - $80,000] -- Yes --> B[Office  
Close to Home]; A -- No --> C([Declined  
Offer]); B -- Yes --> D[Possibility  
to Work Remotely]; B -- No --> E([Declined  
Offer]); D -- Yes --> F[Accepted  
Offer]; D -- No --> G([Declined  
Offer]);
```

The flowchart outlines the decision process for accepting a job offer. It starts with a decision on the salary range (\$50,000 - \$80,000). If the salary is within this range (Yes), the next step is to check if the office is close to home. If the office is close to home (Yes), the next step is to check if there is a possibility to work remotely. If there is a possibility to work remotely (Yes), the offer is accepted. If the office is not close to home (No) or if there is no possibility to work remotely (No), the offer is declined. If the salary is not within the specified range (No), the offer is also declined.

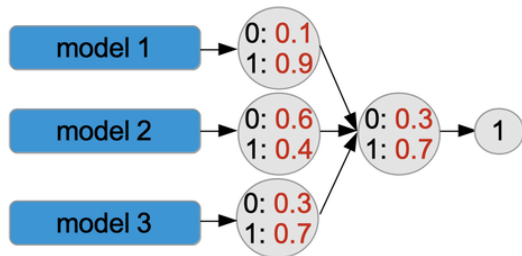
Base element: Decision Tree



Voting: Hard vs Soft



Hard voting



Soft voting

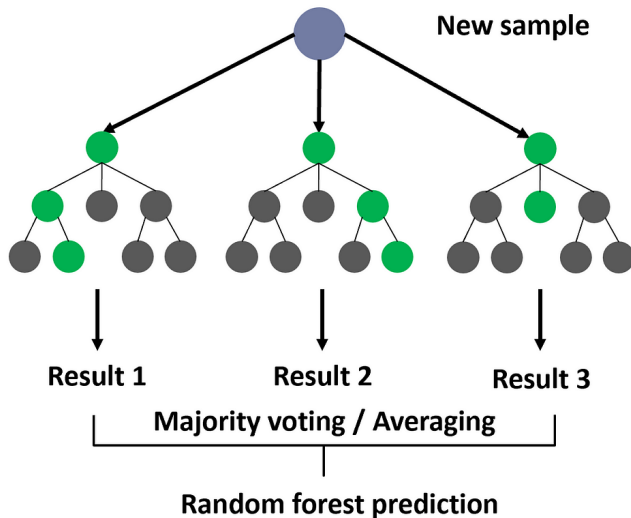
Bootstrap

- Random sampling with replacement
- The basic idea of bootstrapping is that inference about a population from sample data (sample $\rightarrow$ population)
- Why we need it? Make model more robust

Ensemble Models

- Gradient Boosting
- Random Forest

Random Forest



Boosting

$$\hat{y}_i^{(0)} = 0$$

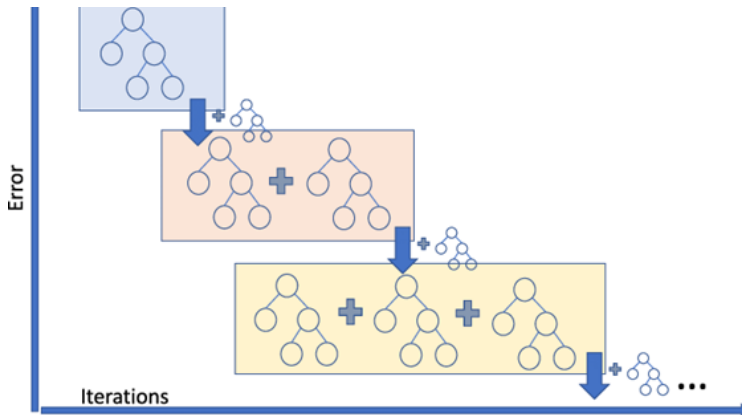
$$\hat{y}_i^{(1)} = f_1(x_i) = \hat{y}_i^{(0)} + f_1(x_i)$$

$$\hat{y}_i^{(2)} = f_1(x_i) + f_2(x_i) = \hat{y}_i^{(1)} + f_2(x_i)$$

...

$$\hat{y}_i^{(t)} = \sum_{k=1}^t f_k(x_i) = \hat{y}_i^{(t-1)} + f_t(x_i)$$

Gradient Boosting



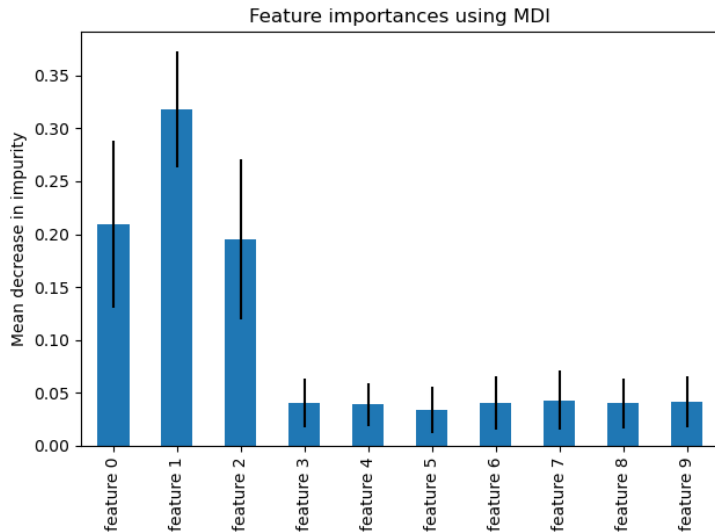
Question

Why not boosting OLS?

Feature Importance of Tree Based Model

- Feature importance can help to identify the most relevant features for a given model and data set.
- There are different methods for calculating feature importance
- Usually computational heavy to calculate, tree-based models comes with handy feature importance

Feature Importance



Three Feature Importance for the Tree Based Model

- Gain Value: gain in term of the Loss function
- Weight Value: weight in term of the leaves
- Cover Value: cover in term of sample

Gain Value

The Gain implies the relative contribution of the corresponding feature to the model calculated by taking each feature's contribution for each tree in the model. A higher value of this metric when compared to another feature implies it is more important for generating a prediction.

Weight Value (or frequency)

The weight value is the percentage representing the relative number of times a particular feature occurs in the trees of the model. In the above example, if feature1 occurred in 2 splits, 1 split and 3 splits in each of tree1, tree2 and tree3; then the weight for feature1 will be $2+1+3 = 6$. The frequency for feature1 is calculated as its percentage weight over weights of all features.

Cover Value

The Cover metric means the relative number of observations related to this feature. For example, if you have 100 observations, 4 features and 3 trees, and suppose feature1 is used to decide the leaf node for 10, 5, and 2 observations in tree1, tree2 and tree3 respectively; then the metric will count cover for this feature as $10+5+2 = 17$ observations. This will be calculated for all the 4 features and the cover will be 17 expressed as a percentage for all features' cover metrics.

Example

Feature	Gain	Cover	Frequency
xxx	2.276101e-01	0.0618490331	1.913283e-02
xxxx	2.047495e-01	0.1337406946	1.373710e-01
xxxx	1.239551e-01	0.1032614896	1.319798e-01
xxxx	6.269780e-02	0.0431682707	1.098646e-01
xxxxx	6.004842e-02	0.0305611830	1.709108e-02

Ensemble Framework

- Voting & Average
- Stacking & Blending

What & Why model blending?

- Use different models to predict one target
- For better performance
- We can see it frequently used by Kaggle.
- Of course, the professional quants use the complicated model blending in their stock, housing, risk model

Model Averaging: Yes just average

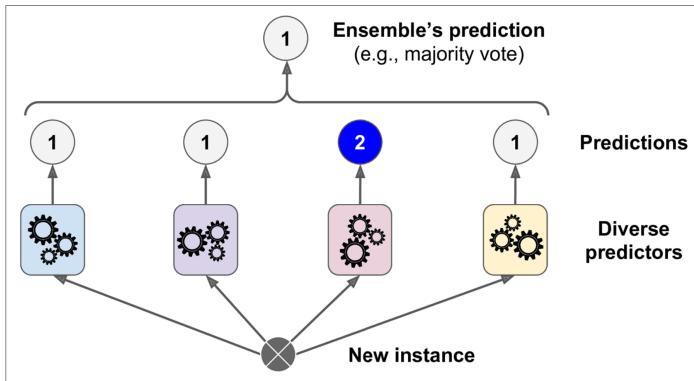


Figure 7-2. **Hard voting** classifier predictions

Model Blending: one model to rule all !!!

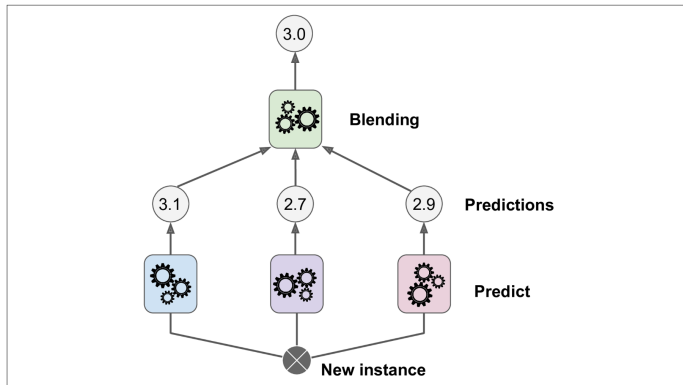
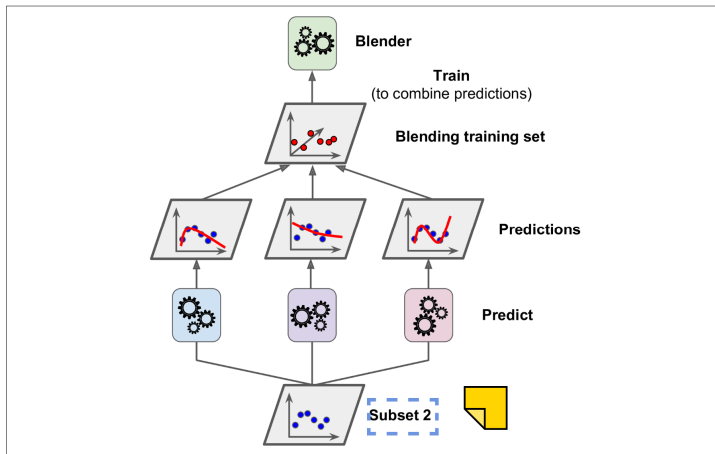


Figure 7-12. Aggregating predictions using a blending predictor

Model Blending: Careful

- you need to split the sample into two part
- one to train sub-models and one to train blender model

Sample Splitting is important



Questions

- Why Sample Splitting is importance
- Problem: Waste huge sample for holdout to training model blender
- Any better way to train model blender?

Ensemble algorithm paper (Chen 2016 XgBoost)

- Read model documentation
- Read original paper

Read model documentation

- Python codes examples
- User guide
- Brief Algorithms
- Xgboost Documentation as an example
(Please read together with me) (Link)

Read original paper

- Detailed Algorithms
- Get connected with other recent top researches
- Math appendix

Good example for your future research

- With this study method, you can also study ANN, RNN, CNN, Keras, TensorFlow, Attention, BERT, ...
- All by yourself efficiently
- XGBoost: A Scalable Tree Boosting System ([Link](#))

Homework: Read XgBoost (Chen 2016)

- XGBoost: A Scalable Tree Boosting System (Tianqi Chen, Carlos Guestrin 2016)
- (Chen Tianqi)
- Cold call questions next class

Reference

1. Hands-on Machine Learning with Scikit-Learn, Keras and TensorFlow (3rd edition)
2. [geeksforgeeks](#)
3. [Kaggle](#)
4. [Wikipedia](#)
5. [ChatGPT](#)
6. [DeepSeek](#)